

CAP: Detecting Network Device Misconfigurations with Context-Aware Prompting of LLMs

Xi Jiang
xijiang9@uchicago.edu
University of Chicago
Chicago, Illinois, USA

Aaron Gember-Jacobson
agemberjacobson@colgate.edu
Colgate University
Hamilton, New York, USA

Nick Feamster
feamster@uchicago.edu
University of Chicago
Chicago, Illinois, USA

Abstract

Traditional tools for verifying network configurations: (1) require high manual effort or lack semantic depth, and (2) lack interpretable, human-readable explanations to support diagnosis. Large Language Models (LLMs) present a promising alternative, but simple approaches like full-file, partition-based, or chain-of-thought prompting fail to handle the large, interconnected nature of configuration files. We introduce the Context-Aware Prompting (CAP) framework, which enables an LLM to reason about network configurations like a human expert. CAP first identifies all potentially relevant neighboring, similar, and referenced configuration segments. CAP then engages the LLM in a structured dialogue, where the model requests the specific context it needs before performing a focused analysis. This on-demand approach provides the necessary context for deep reasoning while avoiding overload. Our evaluation on production network configurations demonstrates that CAP reliably detects a wide range of known and previously unknown configuration errors. CAP outperforms existing LLM-based baselines, and achieves performance comparable to state-of-the-art non-LLM tools.

CCS Concepts

• **Networks** → **Network management**; • **Computing methodologies** → **Machine learning**; • **Security and privacy** → **Network security**.

Keywords

Network anomaly detection, large language model, context retrieval, prompt engineering

ACM Reference Format:

Xi Jiang, Aaron Gember-Jacobson, and Nick Feamster. 2026. CAP: Detecting Network Device Misconfigurations with Context-Aware Prompting of LLMs. In *Abstracts of the 2026 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems (SIGMETRICS Abstracts '26)*, June 08–12, 2026, Ann Arbor, MI, USA. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/3801489.3806917>

1 Motivation

Ensuring the stability, security, and performance of networks requires carefully (re)configuring routers and switches. Network engineers rely on extensive knowledge gained from years of experience with an organization’s network(s), vast vendor documentation [2, 3],

and industry best practices [4]. Furthermore, engineers must consider the interplay between (a change to) a configuration and related aspects of devices’ configurations. This is challenging for humans, because a device’s configuration is often hundreds or thousands of lines long with many dependencies between segments [9].

To address this, prior work has developed two main classes of tools. *Model checkers* [1, 5, 7, 8, 15, 16, 30, 32, 34, 38, 39] verify network-wide properties (e.g., reachability) using formal models of protocols and policies. However, they require significant manual effort to specify these models, which are difficult to construct correctly and completely [10, 38], with limited automation [11, 19, 33]. *Consistency checkers* [1, 14, 17, 18, 21–23, 35] instead flag deviations across devices or from best practices with minimal manual effort, but lack semantic reasoning about intent and complex interactions. Both approaches also provide limited, often hard-to-interpret explanations.

Large language models (LLMs) [6, 13] offer a promising alternative. Pretrained on extensive corpora including protocol specifications and vendor documentation, LLMs can translate configurations into human-readable reasoning about correctness. However, existing approaches either input entire configurations—risking context overload and degraded reasoning [14, 20, 26, 27, 31]—or partition them arbitrarily, which breaks critical dependencies [5, 24, 28, 30].

A key insight is that human engineers do not analyze configurations linearly; instead, they selectively navigate to relevant sections and follow dependencies across the configuration. Effective LLM-based analysis must mimic this behavior by dynamically retrieving relevant context for each segment under inspection. This need to provide precise, context-aware information—capturing both local interactions and long-range dependencies—motivates our Context-Aware Prompting (CAP) framework.

2 Our Framework: Context-Aware Prompting

CAP enables an LLM to analyze network configurations via two integrated components: a **Context Mining** engine that extracts relevant configuration segments for any statement under review, and a **Guided Interactive Prompting** engine that lets the model request only the context it needs before rendering a final verdict. Together, they reframe analysis as context acquisition, allowing the model to iteratively gather only the information required for accurate reasoning.

2.1 Context Mining

Configuration Representation. CAP parses each device’s configuration into a tree T whose nodes represent categories, instances, and parameters. A root-to-leaf path $P = \{\tau \rightarrow V_1 \rightarrow \dots \rightarrow V_k = v_k\}$ constitutes a *fully-qualified configuration statement* (FQCS)—a



This work is licensed under a Creative Commons Attribution 4.0 International License. *SIGMETRICS Abstracts '26*, Ann Arbor, MI, USA
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2570-8/2026/06
<https://doi.org/10.1145/3801489.3806917>

single, unambiguous configuration line enriched with its full hierarchical context. For example, a FQCS can be:

interfaces $\rightarrow$ ge-0/0/1 $\rightarrow$. . . $\rightarrow$ input = DEMO-FILTER.

This representation allows CAP to define context mining as structured tree queries, capturing hierarchical structure, cross-device uniformity, and cross-references. And we define three types of context in this study as follows.

Neighboring Context $N_m(P)$. Neighboring configurations share a common ancestry with P up to depth m :

$$N_m(P) = \{P' \in T \mid V'_i = V_i \text{ for } 1 \leq i \leq m, P' \neq P\}$$

We default to $m = k-2$ (siblings sharing the same parent), capturing the immediate surrounding context without pulling in unrelated higher-level settings.

Similar Context $S_m(P)$. Similar configurations share the same top-level category ($m=1$) and the same terminal parameter (V_k), but differ in intermediate nodes:

$$S_m(P) = \{P' \in T \mid V'_i = V_i \text{ for } 1 \leq i \leq m, V'_k = V_k, P' \neq P\}$$

Matching at $m=1$ (a major functional block) strikes the right balance between breadth and precision, enabling CAP to detect inconsistencies and reason about whether they are intentional.

Referenced Context $R(P)$. Referenced configurations resolve the *definitions* of identifiers named in P . For *intra-device* references, CAP finds all paths where the leaf value v_k appears as an intermediate node:

$$R_{intra}(P) = \{P' \in T \mid v_k \in P' \text{ as an intermediate node}\}$$

Separating user-defined identifiers (e.g., a filter named DEMO-FILTER) from primitive values (e.g., MTU 9000) is non-trivial; CAP delegates this to the LLM, which classifies the value and produces a disambiguation keyword to constrain the search.

For *cross-device* references (e.g., BGP neighbor statements), CAP uses a small rule set to detect inter-device pointers, locates the target device by matching against its identifier set $ID(T_B)$, and retrieves the reciprocal configuration:

$$R_{cross}(P) = \{P' \in T_B \mid \exists id \in ID(T_A) : id \in P' \quad (1)$$

$$\wedge \text{peerkw}(P') = \text{peerkw}(P)\} \quad (2)$$

Requiring both an identifier match and a matching peer keyword ensures only semantically reciprocal statements are retrieved, enabling detection of asymmetric misconfigurations.

Composition. The three primitives compose naturally: $R(N(P))$ retrieves definitions of identifiers neighboring the statement under review, enabling multi-hop reasoning. This flexibility lets CAP emulate an expert's intuition for following dependency chains.

2.2 Guided Interactive Prompting

Rather than supplying all mined context at once—which degrades LLM reasoning even within available token budgets [25]—CAP conducts a two-turn structured dialogue per configuration line.

1. Initial Prompt. The LLM receives only the FQCS under review and a menu of available context categories, and responds with a JSON object specifying which categories it needs and why—simultaneously classifying the terminal value as primitive or user-defined. Requiring an explicit *reason* field creates a transparent audit trail of the model's reasoning.

2. Context Provision & Final Analysis. CAP retrieves exactly the requested context segments and constructs a second prompt. The LLM issues a structured JSON verdict: whether a misconfiguration exists, which parameter is affected, and a natural-language explanation. One well-targeted round is empirically sufficient, avoiding the latency of open-ended iterative loops.

This design decouples context retrieval from reasoning: the LLM curates its own input, making misclassifications attributable to the model's own judgment. The context-request pattern also becomes a diagnostic signal into each model's reasoning strategy.

3 Evaluation

We evaluate CAP around four questions: (1) Can it detect known misconfigurations? (2) Can it find novel issues in production networks? (3) What context do different LLMs request, and how does that affect accuracy? (4) How does it compare to state-of-the-art tools?

Setup. We use configurations from three networks: Internet2 (Juniper, 17 sampled core/aggregation routers, ~18,944 lines avg.), Campus A (Aruba, 8 access switches, ~5,563 lines), and Campus B (Cisco/Juniper, 5 distribution/aggregation routers, ~6,261 lines). We evaluate five LLM backends: GPT-4o, Claude Sonnet 4.5, DeepSeek-V3 (DS), Gemini 2.5 Pro, and LLaMa 4 Maverick.

3.1 Can CAP Detect Known Misconfigurations?

Synthetic misconfigurations (Internet2). We injected 58 misconfigurations across three types—*missing lines*, *incorrect/undefined references*, and *replicated stanzas*—paired with 58 correct updates. Reference errors are most reliably detected (Claude: 84.6% TPR, GPT-4o: 69.2%, FPR <8%), leveraging explicit cross-links as strong relational cues. Replicated-stanza errors follow (70%/60% TPR). Missing-line errors are hardest (32%/36% TPR): absence is inherently ambiguous between omission and intentional design.

Known real-world misconfigurations (Campus A). CAP achieved 100% TPR on all 6 known VLAN-assignment misconfigurations across all five LLM backends. Two of the six had been previously unnoticed despite causing active misrouting.

3.2 Can CAP Identify Novel Issues?

Running CAP over full Campus A configurations uncovered 19 previously unknown issues (2 invalid subnet masks, 14 inconsistent policy names, 1 insecure SSH KEX algorithm, 2 ambiguous rate-limit values), all confirmed by network operators. On Campus B, all 21 flagged configurations were validated as intentional (100% false-positive rate)—including redundant DHCP helpers and undocumented PIM settings—with model behavior diverging sharply: DS flagged only 4 cases while GPT-4o flagged all 21.

3.3 What Context Do LLMs Request?

Models differ systematically in context preferences. GPT-4o and Gemini focus narrowly on direct references—effective when relational cues exist but prone to false positives when intent is ambiguous. Claude and DS consistently request neighboring and similar configurations before deciding, enabling finer-grained reasoning. An ablation confirms the design is efficient: providing all context

upfront reduced GPT-4o's false positives from 21 to 5 but tripled prompt size, with marginal benefit for other models.

3.4 How Does CAP Compare to Existing Tools?

On the Internet2 dataset, CAP achieves 78.4% accuracy (any model), substantially outperforming Ciri [28] (39.7%), full-file prompting (50.0%), and Batfish [15] (54.3%). It matches Diffy [18] (79.3%) overall and exceeds it for reference-based errors (84.6% vs. 69.2%). CAP also detects 5 missing-line cases Diffy cannot surface, since Diffy relies on structural comparison. Critically, unlike any non-LLM tool, CAP produces human-readable reasoning traces, making its outputs actionable for diagnosis and remediation.

4 Conclusion

We present CAP, a framework enabling LLMs to analyze network configurations with expert-level reasoning. CAP addresses context overload and fragmentation via (i) a context mining engine extracting neighboring, similar, and referenced configurations, and (ii) an interactive prompting mechanism letting the model request only needed context. Evaluations show CAP outperforms LLM baselines, matches state-of-the-art non-LLM tools, and generates human-readable reasoning traces that improve operator trust and facilitate remediation.

Acknowledgments

We thank the anonymous associate editors and reviewers for their constructive feedback and suggestions. This work was supported in part by the National Science Foundation under Awards #2334996, #2319603, and #2324515.

References

- [1] Batfish: An open source network configuration analysis tool. <https://batfish.org>.
- [2] Cisco networking software (IOS & NX-OS) - support and downloads. <https://cisco.com/c/en/us/support/ios-nx-os-software>.
- [3] Juniper networks - junos OS documentation. <https://juniper.net/documentation/product/us/en/junos-os>.
- [4] Mutually agreed norms for routing security (MANRS). <https://manrs.org>.
- [5] Anubhavnidhi Abhashkumar, Aaron Gember-Jacobson, and Aditya Akella. Tiramisu: Fast multilayer network verification. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 201–219, 2020.
- [6] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- [7] Ehab Al-Shaer and Mohammed Noraden Alsaleh. Configchecker: A tool for comprehensive security configuration analytics. In *2011 4th Symposium on Configuration Analytics and Automation (SAFECONFIG)*, pages 1–2. IEEE, 2011.
- [8] Ryan Beckett, Aarti Gupta, Ratul Mahajan, and David Walker. A general approach to network configuration verification. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*, pages 155–168, 2017.
- [9] Theophilus Benson, Aditya Akella, and David Maltz. Unraveling the complexity of network management. In *Symposium on Networked Systems Design and Implementation (NSDI)*, 2009.
- [10] Rudiger Birkner, Tobias Brodmann, Petar Tsankov, Laurent Vanbever, and Martin Vechev. Metha: Network verifiers need to be correct too! In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2021.
- [11] Rudiger Birkner, Dana Drachler-Cohen, Laurent Vanbever, and Martin T. Vechev. Config2spec: Mining network specifications from network configurations. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2020.
- [12] Mark Bogdanov. *Leveraging Advanced Large Language Models To Optimize Network Device Configuration*. PhD thesis, Purdue University Graduate School, 2024.
- [13] DeepSeek-AI. Deepseek-v3 technical report, 2024.
- [14] Nick Feamster and Hari Balakrishnan. Detecting bgp configuration faults with static analysis. In *Proceedings of the 2nd conference on Symposium on Networked Systems Design & Implementation-Volume 2*, pages 43–56, 2005.
- [15] Ari Fogel, Stanley Fung, Luis Pedrosa, Meg Walraed-Sullivan, Ramesh Govindan, Ratul Mahajan, and Todd Millstein. A general approach to network configuration analysis. In *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*, pages 469–483, 2015.
- [16] Alan Jeffrey and Taghrid Samak. Model checking firewall policy configurations. In *2009 IEEE international symposium on policies for distributed systems and networks*, pages 60–67. IEEE, 2009.
- [17] Siva Kesava Reddy Kakarla, Alan Tang, Ryan Beckett, Karthick Jayaraman, Todd Millstein, Yuval Tamir, and George Varghese. Finding network misconfigurations by automatic template inference. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 999–1013, 2020.
- [18] Siva Kesava Reddy Kakarla, Francis Y Yan, and Ryan Beckett. Diffy: Data-driven bug finding for configurations. *Proceedings of the ACM on Programming Languages*, 8(PLDI):199–222, 2024.
- [19] Ali Kheradmand. Automatic inference of high-level network intents by mining forwarding patterns. In *Symposium on SDN Research (SOSR)*, 2020.
- [20] Hoyoon Kim, Theophilus Benson, Aditya Akella, and Nick Feamster. The evolution of network configuration: a tale of two campuses. In *Proceedings of the 11th ACM SIGCOMM Internet Measurement Conference (IMC)*, 2011.
- [21] Franck Le, Sihyung Lee, Tina Wong, Hyong S Kim, and Darrell Newcomb. Characterization and problem detection of routing policy configurations. 2006.
- [22] Franck Le, Sihyung Lee, Tina Wong, Hyong S Kim, and Darrell Newcomb. Minerals: using data mining to detect router misconfigurations. In *Proceedings of the 2006 SIGCOMM workshop on Mining network data*, pages 293–298, 2006.
- [23] Franck Le, Sihyung Lee, Tina Wong, Hyong S Kim, and Darrell Newcomb. Detecting network-wide and router-specific misconfigurations through data mining. *IEEE/ACM transactions on networking*, 17(1):66–79, 2008.
- [24] Franck Le, Geoffrey G. Xie, and Hui Zhang. Understanding route redistribution. In *2007 IEEE International Conference on Network Protocols (ICNP)*, 2007.
- [25] Mosh Levy, Alon Jacoby, and Yoav Goldberg. Same task, more tokens: the impact of input length on the reasoning performance of large language models. *arXiv preprint arXiv:2402.14848*, 2024.
- [26] Jiaqi Li, Mengmeng Wang, Zilong Zheng, and Muhang Zhang. Can long-context large language models understand long contexts?
- [27] Tianle Li, Ge Zhang, Quy Duc Do, Xiang Yue, and Wenhui Chen. Long-context llms struggle with long in-context learning. *arXiv preprint arXiv:2404.02060*, 2024.
- [28] Xinyu Lian, Yinfang Chen, Runxiang Cheng, Jie Huang, Parth Thakkar, and Tianyin Xu. Configuration validation with large language models. *arXiv preprint arXiv:2310.09690*, 2023.
- [29] Chang Liu, Xiaohui Xie, Xinggong Zhang, and Yong Cui. Large language models for networking: Workflow, advances and challenges. *IEEE Network*, 2024.
- [30] Santhosh Prabhu, Kuan Yen Chou, Ali Kheradmand, Brighton Godfrey, and Matthew Caesar. Plankton: Scalable network configuration verification through model checking. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 953–967, 2020.
- [31] Hongjin Qian, Zheng Liu, Peitian Zhang, Kelong Mao, Yujia Zhou, Xu Chen, and Zhicheng Dou. Are long-llms a necessity for long-context tasks? *arXiv preprint arXiv:2405.15318*, 2024.
- [32] Ronald W Ritchey and Paul Ammann. Using model checking to analyze network vulnerabilities. In *Proceeding 2000 IEEE Symposium on Security and Privacy. S&P 2000*, pages 156–165. IEEE, 2000.
- [33] Prakhar Sharma and Vinod Yegneswaran. Prosper: Extracting protocol specifications using large language models. In *Proceedings of the 22nd ACM Workshop on Hot Topics in Networks (HotNets)*, 2023.
- [34] Samuel Steffen, Timon Gehr, Petar Tsankov, Laurent Vanbever, and Martin T. Vechev. Probabilistic verification of network configurations. In *SIGCOMM*, 2020.
- [35] Alan Tang, Siva Kesava Reddy Kakarla, Ryan Beckett, Ennan Zhai, Matt Brown, Todd D. Millstein, Yuval Tamir, and George Varghese. Campion: debugging router configuration differences. In *ACM SIGCOMM 2021 Conference*, 2021.
- [36] Changjie Wang, Mariano Scazzariello, Alireza Farshin, Simone Ferlin, Dejan Kostić, and Marco Chiesa. NetConfEval: Can LLMs facilitate network configuration? *Proceedings of the ACM on Networking*, 2(CoNEXT2):1–25, 2024.
- [37] Zehao Wang, Dong Jae Kim, and Tse-Hsun Chen. Identifying performance-sensitive configurations in software systems through code analysis with llm agents. *arXiv preprint arXiv:2406.12806*, 2024.
- [38] Fangdan Ye, Da Yu, Ennan Zhai, Hongqiang Harry Liu, Bingchuan Tian, Qiaobo Ye, Chunsheng Wang, Xin Wu, Tianchen Guo, Cheng Jin, Duncheng She, Qing Ma, Biao Cheng, Hui Xu, Ming Zhang, Zhiliang Wang, and Rodrigo Fonseca. Accuracy, scalability, coverage: A practical configuration verifier on a global WAN. In *SIGCOMM*, 2020.
- [39] Peng Zhang, Dan Wang, and Aaron Gember-Jacobson. Symbolic router execution. In Fernando Kuipers and Ariel Orda, editors, *ACM SIGCOMM Conference*, 2022.